Minza's Complete Learning Book

From Anaconda Setup to Ranking #1 on Google

A Structured Guide for a Class 12 Student
Building a Professional Website for His Father's
Tax Consultancy & Advocacy Practice

Author: Minza

Platform: Windows + Anaconda + Flask

Compiled on: Overleaf

Note

This book is your personal roadmap. Every chapter builds on the last. Work through it week by week, chapter by chapter. By the end, your father's practice will have a professional website, strong Google rankings, and a growing online presence — built entirely by **you**.

Contents

Introduction: Before You Begin **7**

1 Setting Up Anaconda Properly on Windows **9**
1.1 What is Anaconda and Why Are We Using It? 9
1.2 Verifying Your Anaconda Installation 9
1.3 Creating a Dedicated Virtual Environment 10
1.4 Installing All Required Packages 10
1.4.1 Installing via pip (inside the webdev environment) 11
1.4.2 Why Not Use conda install for These? 12
1.4.3 Installing pandas via conda (better on Windows) 12
1.5 Verifying All Packages Are Installed 12
1.6 Setup Checklist 13
1.7 Common Beginner Mistakes with Anaconda on Windows 14

2 Building Your Father's Professional Website with Flask **17**
2.1 Why Flask and Not Django? 17
2.2 Understanding How a Flask Website Works 17
2.3 Project Folder Structure 18
2.4 Creating the Flask Application (`app.py`) 18
2.5 Creating the Base HTML Template 21
2.6 Creating the Home Page (`index.html`) 23
2.7 Creating the About Page (`about.html`) 25
2.8 Creating the Services Page (`services.html`) 26
2.9 Creating the Contact Page (`contact.html`) 28
2.10 Adding CSS Styling (`static/css/style.css`) 29
2.11 Running Your Website Locally 35
2.12 Deploying Your Website for Free on Render 35
2.12.1 Step 1: Create requirements.txt 35
2.12.2 Step 2: Create a Procfile 36
2.12.3 Step 3: Push to GitHub and Connect to Render 36

3 SEO: Ranking #1 in Your City on Google **39**
3.1 What is SEO and How Does Google Decide Rankings? 39
3.2 Keyword Research: Finding the Right Words to Target 39
3.2.1 Types of Keywords 40
3.2.2 Free Keyword Research Tools 40
3.2.3 Your Target Keywords List 40
3.3 On-Page SEO 40
3.3.1 Title Tags 41

3.3.2 Meta Descriptions . . . 41
3.3.3 Heading Structure (H1–H6) . . . 42
3.3.4 Image Alt Text . . . 42
3.3.5 Keyword Placement Checklist . . . 42
3.4 Technical SEO . . . 42
3.4.1 Creating sitemap.xml . . . 43
3.4.2 Creating robots.txt . . . 43
3.4.3 Page Speed Optimisation . . . 44
3.5 Local SEO: Dominating Your City . . . 44
3.5.1 Google Business Profile (GBP) . . . 44
3.5.2 NAP Consistency . . . 45
3.5.3 Getting Google Reviews . . . 45
3.5.4 Local Citations . . . 45
3.6 Schema Markup for Rich Results . . . 46
3.7 Setting Up Google Search Console . . . 46
3.8 Complete SEO Checklist . . . 47

4 Managing Your Father's LinkedIn Profile 51
4.1 Why LinkedIn Matters for a Tax Consultant . . . 51
4.2 Optimising the LinkedIn Profile . . . 51
4.2.1 Profile Photo and Banner . . . 51
4.2.2 Headline . . . 51
4.2.3 About Section (Summary) . . . 52
4.2.4 Experience Section . . . 52
4.2.5 Skills Section . . . 52
4.2.6 Featured Section . . . 52
4.3 LinkedIn SEO: Ranking in LinkedIn Search . . . 52
4.4 Content Strategy and Calendar . . . 53
4.4.1 Post Frequency . . . 53
4.4.2 Content Types (with Examples) . . . 53
4.4.3 Monthly Content Calendar Template . . . 53
4.5 Growing the Professional Network . . . 54
4.6 Metrics to Track . . . 54

5 Python Automation for SEO and Web Management 57
5.1 Introduction: Why Automate? . . . 57
5.2 Script 1: Keyword Ranking Tracker . . . 57
5.3 Script 2: On-Page SEO Auditor . . . 60
5.4 Script 3: Sitemap Generator . . . 65
5.5 Script 4: Website Uptime Monitor . . . 67
5.6 Script 5: Running Scripts on a Schedule (Windows) . . . 69

6 12-Week Learning and Execution Roadmap 73
6.1 How to Use This Roadmap . . . 73
6.2 Week 1 – Anaconda, Python, and Your First Flask Page . . . 73
6.3 Week 2 – HTML and CSS Fundamentals . . . 74
6.4 Week 3 – Flask Templates and All Four Pages . . . 74
6.5 Week 4 – Deploying to the Internet . . . 74
6.6 Week 5 – On-Page SEO Implementation . . . 74

6.7 Week 6 – Technical SEO and Google Search Console 75
6.8 Week 7 – Local SEO: Google Business Profile 75
6.9 Week 8 – LinkedIn Profile Setup 75
6.10 Week 9 – Python SEO Automation: Auditor and Sitemap 76
6.11 Week 10 – Keyword Tracking and Uptime Monitoring 76
6.12 Week 11 – LinkedIn Content and Community 76
6.13 Week 12 – Review, Improve, and Plan Ahead 77
6.14 After Week 12: Continuing the Journey 77

Conclusion **79**

Introduction: Before You Begin

Welcome, Minza! You are a Class 12 student with basic knowledge of Java and Python, and you have just installed **Anaconda** on your Windows laptop. That is a great starting point. This book is written entirely for *you* — not for professionals, not for experts, but for a smart beginner who wants to do real, meaningful work.

Here is what this book will help you build, step by step:

- A **professional website** for your father's Tax Consultancy and Advocacy practice.
- Strong **Google rankings** so people in your city find him first.
- An optimised **LinkedIn profile** that builds his professional reputation.
- **Python automation scripts** that do the boring SEO work for you.
- A clear **12-week plan** so you never feel lost about what to do next.

Note

How to use this book: Read each chapter in order. Do not skip ahead. Each chapter assumes you have finished the previous one. Every chapter ends with a *What You Learned* box and a *Practice Task*. Always do the practice task before moving on — that is how real learning happens.

Chapter 1

Setting Up Anaconda Properly on Windows

1.1 What is Anaconda and Why Are We Using It?

Before typing a single command, you need to understand *why* we are using Anaconda.

Python is the programming language we will use for almost everything in this book — building the website, writing automation scripts, doing SEO audits.

Anaconda is not Python itself. It is a *distribution* of Python — meaning it comes with Python already included, plus hundreds of useful libraries pre-installed, plus a powerful tool called **conda** that helps you manage your projects cleanly.

> **Note**
>
> **Simple analogy:** Python is like electricity. Anaconda is like a complete electrical kit — it gives you the wires, switches, and a control panel (conda) all in one box, so you do not have to find each piece separately.

The most important feature of Anaconda for us is **virtual environments**. A virtual environment is like a separate, clean room for each project. Each room has its own set of tools (Python packages) that do not interfere with tools in other rooms. This keeps your projects safe and organised.

1.2 Verifying Your Anaconda Installation

Open the **Anaconda Prompt** (search for it in the Windows Start Menu — do *not* use the regular Command Prompt for this). Then type these commands one by one.

```
# Check which version of conda is installed
conda --version

# Check which version of Python is installed
python --version

```

```
# See a list of all conda environments (right now there is only 'base')
conda env list

# See all packages currently installed in the base environment
conda list
```

Listing 1.1: Verify Anaconda Installation

Tip

Expected output for `conda -version`: something like `conda 23.x.x` or higher. **Expected output for** `python -version`: something like `Python 3.10.x` or higher. If you see an error like `conda is not recognised`, Anaconda was not added to your PATH correctly. Re-run the Anaconda installer and tick the box that says *"Add Anaconda to my PATH environment variable"*.

1.3 Creating a Dedicated Virtual Environment

We will create one virtual environment called `webdev` that we will use for *all* the work in this book. This keeps everything organised in one place.

```
# Create a new environment named 'webdev' with Python 3.11
# (3.11 is stable and well-supported as of 2024)
conda create --name webdev python=3.11

# When it asks "Proceed ([y]/n)?", type y and press Enter

# Now activate (enter) the environment
conda activate webdev

# Your prompt should now show: (webdev) C:\Users\YourName>
# That (webdev) prefix means you are inside the environment.
```

Listing 1.2: Create the webdev Environment

Warning

Always activate your environment before working! Every time you open a new Anaconda Prompt, you must type `conda activate webdev` before running any Python or pip command. Forgetting this is the #1 beginner mistake. If you install a package without activating, it goes into the wrong place and your project will break.

1.4 Installing All Required Packages

Now we will install every package you need for this entire book, all at once. Read the explanation for each package — understanding *why* you need it is just as important

as installing it.

1.4.1 Installing via pip (inside the webdev environment)

```
# Make sure (webdev) is showing in your prompt before running this!

# Flask     the web framework for building your father's website.
# Think of it as the engine that powers your website.
pip install flask

# Jinja2     Flask uses this automatically for HTML templates.
# It lets you insert Python variables into your HTML pages.
pip install jinja2

# Requests     lets Python send HTTP requests (like a browser visiting a URL).
# We use this in our SEO scripts to fetch web pages for analysis.
pip install requests

# BeautifulSoup4     a web scraping library.
# It reads HTML and lets you extract specific information from it,
# like all the headings, links, or meta tags on a page.
pip install beautifulsoup4

# lxml     a fast HTML/XML parser that BeautifulSoup uses under the hood.
pip install lxml

# schedule     lets you run Python scripts automatically at set times,
# like a cron job but written in pure Python. Great for Windows.
pip install schedule

# python-dotenv     lets you store secret settings (like passwords, API keys)
# in a separate file (.env) instead of writing them in your code.
pip install python-dotenv

# gunicorn     a production-grade web server for Flask apps.
# Used when deploying your website to the internet.
# Note: gunicorn does not work natively on Windows,
# but we install it now for deployment on Render (Linux server).
pip install gunicorn
```

```

# uptime-related: we use requests for this, already installed above.

# pandas      powerful data analysis library.
# We use it to store and analyse keyword ranking data over time.
pip install pandas

# openpyxl      lets pandas save data to Excel files (.xlsx).
pip install openpyxl
```

Listing 1.3: Install All Required Packages

1.4.2 Why Not Use conda install for These?

Note

conda install vs pip install — when to use which?
conda install is preferred for:

- Scientific/data science packages (numpy, pandas, scipy, matplotlib)
- Packages that need compiled C/C++ code to work on Windows
- When you want the most stable, tested version

pip install is preferred for:

- Web development packages (Flask, requests, beautifulsoup4)
- Packages not available in the conda repository
- Smaller utility packages

Rule of thumb: Always try `conda install` first. If the package is not found, use `pip install`. Never mix them carelessly — once you use pip in a conda environment, continue using pip for packages in that environment to avoid conflicts.

1.4.3 Installing pandas via conda (better on Windows)

```
# pandas is better installed via conda on Windows
# because conda handles the compiled dependencies automatically
conda install pandas openpyxl
```

Listing 1.4: Install pandas via conda

1.5 Verifying All Packages Are Installed

```
# List all installed packages in your webdev environment
conda list

# Test that Flask works by importing it in Python
python -c "import flask; print('Flask version:', flask.
    __version__)"

# Test that BeautifulSoup works
python -c "import bs4; print('BeautifulSoup version:', bs4.
    __version__)"

# Test that requests works
python -c "import requests; print('Requests version:',
    requests.__version__)"

# Test that pandas works
python -c "import pandas; print('Pandas version:', pandas.
    __version__)"
```

Listing 1.5: Verify Installations

If each command prints a version number without errors, your setup is complete.

1.6 Setup Checklist

Setup Checklist

Tick each box once completed:

- ☐ Anaconda Prompt opens correctly on Windows
- ☐ `conda -version` returns a version number
- ☐ `python -version` returns Python 3.10 or higher
- ☐ `conda create -name webdev python=3.11` ran successfully
- ☐ `conda activate webdev` works and shows `(webdev)` in prompt
- ☐ Flask is installed and importable
- ☐ BeautifulSoup4 is installed and importable
- ☐ requests is installed and importable
- ☐ pandas is installed and importable
- ☐ schedule, python-dotenv, openpyxl are installed

1.7 Common Beginner Mistakes with Anaconda on Windows

1. **Forgetting to activate the environment.** Always run `conda activate webdev` first. Check that your prompt shows `(webdev)`.
2. **Using the wrong terminal.** Always use **Anaconda Prompt**, not Windows PowerShell or Command Prompt, unless you have specifically configured those to recognise conda.
3. **Installing packages in base.** The `base` environment is Anaconda's home environment. Never install project packages there. Always use `webdev` (or create a new env per project).
4. **Mixing pip and conda incorrectly.** If you install a package with conda and then try to upgrade it with pip, you can break the environment. Decide early: for web dev, use pip. For data science tools, use conda.
5. **Closing the terminal and losing your environment.** The files are saved — but you must re-run `conda activate webdev` every time you open a new terminal. Your work files are in your folder, not the terminal.

What You Learned

- Anaconda is a Python distribution that includes conda for environment management.
- A virtual environment keeps each project's packages isolated and safe.
- We created a dedicated environment called `webdev`.
- We installed Flask, BeautifulSoup4, requests, pandas, schedule, and other tools.
- The golden rule: always activate your environment before working.
- conda install is best for data-science packages; pip install for web packages.

Practice Task

Open your Anaconda Prompt and do the following without looking at the book:

1. Activate your `webdev` environment.
2. Check the version of Flask you installed.
3. List all installed packages with `conda list` and count how many there are.
4. Type `python` to enter the Python shell, then type `import flask` and press Enter. If no error appears, Flask is ready. Type `exit()` to leave

the shell.

Chapter 2

Building Your Father's Professional Website with Flask

2.1 Why Flask and Not Django?

You asked which framework to use. Here is the honest answer.

Django is a full-featured web framework. It includes a built-in admin panel, user authentication, a database layer, and much more out of the box. It is excellent — but it is a lot to learn all at once for a beginner.

Flask is a *micro-framework*. It gives you just the essentials, and you add only what you need. For a professional business website (a few pages, a contact form, no complex user accounts needed), Flask is **perfect**. Less to learn, faster to build, easier to understand.

> **Note**
>
> **Think of it this way:** Django is a fully furnished flat. Flask is an empty room with great walls — you decide what furniture to add. For your father's 4-page website, an empty room is all you need.

2.2 Understanding How a Flask Website Works

Before writing any code, understand the flow:

1. A visitor types `www.yourfather.com` in their browser.
2. The browser sends an HTTP request to your Flask application.
3. Flask reads the URL (e.g., `/about`) and matches it to a Python function.
4. That Python function returns an HTML page.
5. The browser displays the HTML as a webpage.

The Python functions that handle URLs are called **routes**. The HTML files are called **templates**. Flask uses a templating language called **Jinja2** to fill dynamic data into your HTML templates (like your father's name, phone number, etc.).

2.3 Project Folder Structure

Create this folder structure on your computer. Open File Explorer and create the folders manually, or use the commands below.

```
# Navigate to your Desktop (or wherever you want to work)
cd C:\Users\YourName\Desktop

# Create the main project folder
mkdir fathers_website

# Move into it
cd fathers_website

# Create the subfolders
mkdir templates
mkdir static
mkdir static\css
mkdir static\images
```

Listing 2.1: Create Project Folders

Your final structure should look like this:

```
fathers_website/
    app.py                  <-- main Python file (the Flask app)
    requirements.txt        <-- list of packages the app needs
    .env                    <-- secret settings (never share this)
    templates/
        base.html           <-- master template all pages inherit from
        index.html          <-- Home page
        about.html          <-- About page
        services.html       <-- Services page
        contact.html        <-- Contact page
    static/
        css/
            style.css       <-- your custom CSS styles
        images/             <-- put logo and photos here
```

Listing 2.2: Project Structure

2.4 Creating the Flask Application (app.py)

```
#         app.py


# This is the heart of your Flask website.
```

```python
# It defines all the URL routes and what HTML page each one shows.

from flask import Flask, render_template, request, flash, redirect, url_for
# Flask        : the main class that creates our web application
# render_template : loads an HTML file from the templates/ folder
# request      : lets us read data submitted in forms (like the contact form)
# flash        : shows one-time messages to the user (e.g., "Message sent!")
# redirect     : sends the user to a different URL
# url_for      : generates URLs for our routes by name (safer than hardcoding)

import os
# os : lets us read environment variables (secret settings from .env file)

from dotenv import load_dotenv
# load_dotenv : reads the .env file and makes its values available to os.getenv()

# Load environment variables from the .env file
load_dotenv()

# Create the Flask application instance
# __name__ tells Flask where to look for templates and static files
app = Flask(__name__)

# Set a secret key      Flask needs this to use the flash() feature securely.
# In production, store this in your .env file, not here directly.
app.secret_key = os.getenv('SECRET_KEY', 'change-this-in-production')

#         ROUTES


# A "route" connects a URL path to a Python function.
# When a visitor goes to that URL, Flask calls the function
# and returns whatever the function gives back (usually an HTML page).

@app.route('/')                 # The home page URL: www.yoursite.com/
def index():
```

```
    """Home page of the website."""
    # render_template loads templates/index.html and sends it to the browser
    return render_template('index.html')

@app.route('/about')            # www.yoursite.com/about
def about():
    """About page     father's background and qualifications."""
    return render_template('about.html')

@app.route('/services')         # www.yoursite.com/services
def services():
    """Services page     list of services offered."""
    return render_template('services.html')

@app.route('/contact', methods=['GET', 'POST'])
# methods=['GET', 'POST'] means this route handles both:
#   GET  : visitor just loads the contact page (browser requests the form)
#   POST : visitor submits the contact form (browser sends the form data)
def contact():
    """Contact page with enquiry form."""
    if request.method == 'POST':
        # Read the values the visitor typed into the form
        name    = request.form.get('name')     # The 'name' input field
        email   = request.form.get('email')    # The 'email' input field
        message = request.form.get('message')  # The 'message' textarea

        # For now, we just print to the console so you can see it working.
        # In a real deployment, you would send an email here using smtplib.
        print(f"--- New Contact Enquiry ---")
        print(f"Name    : {name}")
        print(f"Email   : {email}")
        print(f"Message: {message}")
        print(f"---------------------------")

        # Show a success message to the visitor
        flash('Thank you for your message! We will contact you shortly.', 'success')

        # Redirect back to the contact page (clears the form)
        return redirect(url_for('contact'))

    # If it is a GET request, just show the contact form
```

```
    return render_template('contact.html')

#         RUN THE APP


if __name__ == '__main__':
    # debug=True means Flask will:
    #   1. Show detailed error messages in the browser
    #   2. Auto-reload when you save changes to app.py
    # NEVER use debug=True in production (on a live server)
    app.run(debug=True)
```

Listing 2.3: app.py – The Main Flask Application

2.5 Creating the Base HTML Template

All our pages will *inherit* from one master template called `base.html`. This means the navigation bar, header, and footer are written *once* in base.html, and every other page automatically gets them. If you change the navigation, you only change it in one file.

```
<!DOCTYPE html>
<!-- DOCTYPE tells the browser this is a modern HTML5 document -->
<html lang="en">

<head>
    <!-- The <head> contains invisible settings, not visible content -->
    <meta charset="UTF-8">
    <!-- charset=UTF-8 supports all languages including Hindi/regional scripts -->

    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <!-- viewport makes the website work correctly on mobile phones -->

    <!--         SEO Meta Tags         -->
    <!-- These tags tell Google what your page is about -->
    <title>{% block title %}Tax Consultant & Advocate{% endblock %} | [Your City]</title>
    <!-- {% block title %} is Jinja2 syntax. Each page can override this block
         to set its own title. If it doesn't, the default above is used. -->

    <meta name="description"
          content="{% block description %}Expert Tax Consultant and Advocate in [Your City].
```

```
            Specialising in Income Tax, GST, and Legal Services.{% endblock %}">
    <!-- The description appears under your link in Google search results.
         Each page should have a unique, relevant description. -->

    <!--        Stylesheet        -->
    <link rel="stylesheet"
          href="{{ url_for('static', filename='css/style.css') }}">
    <!-- url_for() generates the correct URL to your CSS file.
         This is safer than hardcoding the path. -->
</head>

<body>

    <!--        Navigation Bar        -->
    <nav class="navbar">
        <div class="nav-brand">
            <!-- Your father's name as the site brand -->
            <a href="{{ url_for('index') }}">[Father's Name]</a>
            <!-- url_for('index') generates the URL for the index() route in app.py -->
        </div>
        <ul class="nav-links">
            <li><a href="{{ url_for('index') }}">Home</a></li>
            <li><a href="{{ url_for('about') }}">About</a></li>
            <li><a href="{{ url_for('services') }}">Services</a></li>
            <li><a href="{{ url_for('contact') }}">Contact</a></li>
        </ul>
    </nav>

    <!--        Flash Messages        -->
    <!-- Shows any messages created by flash() in app.py -->
    {% with messages = get_flashed_messages(with_categories=true) %}
        {% if messages %}
            {% for category, message in messages %}
                <div class="alert alert-{{ category }}">{{ message }}</div>
            {% endfor %}
        {% endif %}
    {% endwith %}

```

```
    <!--        Main Content Block        -->
    <!-- Each child template fills in this block with its own content -->
    <main class="main-content">
        {% block content %}{% endblock %}
    </main>

    <!--        Footer        -->
    <footer class="footer">
        <div class="footer-content">
            <p><strong>[Father's Name]</strong>     Tax Consultant &amp; Advocate</p>
            <p>       [Your City, State]  | 
                      [Phone Number]  | 
                      [Email Address]</p>
            <p class="footer-copy">&copy; 2025 [Father's Name]. All rights reserved.</p>
        </div>
    </footer>

</body>
</html>
```

Listing 2.4: templates/base.html

2.6 Creating the Home Page (index.html)

```
{% extends "base.html" %}
<!-- This line means: "use base.html as the master layout for this page" -->

{% block title %}Home{% endblock %}
<!-- Sets the page title to: "Home | [Your City]" -->

{% block description %}
Professional Tax Consultant and Advocate in [Your City].
Expert in Income Tax, GST Filing, and Legal Advisory Services.
Call today for a free consultation.
{% endblock %}

{% block content %}
<!-- Everything inside this block goes into the {% block content %} in base.html -->

<!--        Hero Section        -->
<section class="hero">
    <div class="hero-text">
        <h1>Expert Tax &amp; Legal Services in [Your City]</h1>
```

```
        <!-- h1 is the most important heading on the page.
             Put your main keyword here     Google reads this first. -->
        <p class="hero-subtitle">
            Trusted Tax Consultant and Advocate with [X] years of experience.
            Helping individuals and businesses with Income Tax, GST, and Legal matters.
        </p>
        <a href="{{ url_for('contact') }}" class="btn-primary">
            Book a Free Consultation
        </a>
    </div>
</section>

<!--        Introduction Section        -->
<section class="intro section-pad">
    <h2>Why Choose [Father's Name]?</h2>
    <!-- h2 is the second-level heading. Use it for major sections. -->
    <div class="features-grid">
        <div class="feature-card">
            <h3>Income Tax Filing</h3>
            <!-- h3 is third-level. Good for card titles and sub-sections. -->
            <p>Accurate and timely filing for salaried individuals, businesses,
                and NRIs. Maximise your refunds legally.</p>
        </div>
        <div class="feature-card">
            <h3>GST Registration &amp; Returns</h3>
            <p>Complete GST compliance for your business. Registration,
                monthly/quarterly returns, and audit support.</p>
        </div>
        <div class="feature-card">
            <h3>Legal Advisory</h3>
            <p>Professional legal advice on taxation disputes, appeals before
                Income Tax Appellate Tribunal (ITAT), and more.</p>
        </div>
    </div>
</section>

<!--        Call to Action        -->
<section class="cta section-pad">
    <h2>Ready to Simplify Your Taxes?</h2>
```

```
    <p>Located in [Your City]. Serving clients across [Your Region] for [X] years.</p>
    <a href="{{ url_for('contact') }}" class="btn-primary">Get in Touch Today</a>
</section>

{% endblock %}
```

Listing 2.5: templates/index.html

2.7 Creating the About Page (about.html)

```
{% extends "base.html" %}

{% block title %}About{% endblock %}

{% block description %}
Learn about [Father's Name], a qualified Tax Consultant and Advocate
based in [Your City] with [X] years of experience in Income Tax and GST.
{% endblock %}

{% block content %}

<section class="page-header">
    <h1>About [Father's Name]</h1>
    <p>Qualified. Experienced. Trusted.</p>
</section>

<section class="about-content section-pad">
    <div class="about-grid">
        <div class="about-text">
            <h2>Professional Background</h2>
            <p>
                [Father's Name] is a qualified <strong>Tax Consultant and Advocate</strong>
                practising in [Your City] for over [X] years.
                He holds [Qualifications, e.g., B.Com, LLB, CA Inter] and is a member of
                the [Bar Council / Professional Body].
            </p>
            <p>
                His practice covers a wide range of services including
                <strong>Income Tax filing and planning</strong>,
                <strong>GST compliance</strong>, and
                <strong>legal representation</strong> before tax authorities and tribunals.
```

```
            </p>

            <h2>Qualifications</h2>
            <ul>
                <!-- Replace with your father's actual qualifications -->
                <li>[Degree, e.g., Bachelor of Commerce (B.Com)]</li>
                <li>[Degree, e.g., Bachelor of Laws (LLB)]</li>
                <li>[Membership, e.g., Member, Bar Council of [State]]</li>
                <li>[Membership, e.g., Registered Income Tax Practitioner]</li>
            </ul>
        </div>
    </div>
</section>

{% endblock %}
```

Listing 2.6: templates/about.html

2.8 Creating the Services Page (services.html)

```
{% extends "base.html" %}

{% block title %}Services{% endblock %}

{% block description %}
Comprehensive tax and legal services in [Your City]: Income Tax Filing,
GST Registration, Tax Planning, Legal Advisory, and more.
{% endblock %}

{% block content %}

<section class="page-header">
    <h1>Our Services</h1>
    <p>Comprehensive Tax &amp; Legal Solutions for Individuals and Businesses</p>
</section>

<section class="services-list section-pad">
    <!-- Each service is a card with a heading and description.
         The headings also serve as SEO keywords. -->

    <div class="service-card">
        <h2>Income Tax Filing</h2>
```

```
        <p>ITR filing for salaried individuals, self-employed professionals,
            partnership firms, and companies. Timely, accurate, and compliant.</p>
    </div>

    <div class="service-card">
        <h2>GST Registration &amp; Returns</h2>
        <p>New GST registration, GSTIN compliance, monthly/quarterly return
            filing (GSTR-1, GSTR-3B), and annual return filing (GSTR-9).</p>
    </div>

    <div class="service-card">
        <h2>Tax Planning &amp; Advisory</h2>
        <p>Strategic advice to legally minimise your tax liability.
            Includes investment planning under Section 80C, HRA, and more.</p>
    </div>

    <div class="service-card">
        <h2>Legal Representation</h2>
        <p>Representation before the Assessing Officer, Commissioner of
            Income Tax (Appeals), and Income Tax Appellate Tribunal (ITAT).</p>
    </div>

    <div class="service-card">
        <h2>TDS &amp; TCS Compliance</h2>
        <p>TDS deduction, deposit, and return filing. Correction statements
            and TDS certificates (Form 16/16A).</p>
    </div>

    <div class="service-card">
        <h2>Business Registration</h2>
        <p>Partnership firm registration, LLP formation, company incorporation,
            and trade licence assistance.</p>
    </div>
</section>

{% endblock %}
```

Listing 2.7: templates/services.html

2.9 Creating the Contact Page (contact.html)

```
{% extends "base.html" %}

{% block title %}Contact{% endblock %}

{% block description %}
Contact [Father's Name], Tax Consultant and Advocate in [Your City].
Call or email for an appointment. Located at [Address].
{% endblock %}

{% block content %}

<section class="page-header">
    <h1>Contact Us</h1>
    <p>We are here to help. Reach out for a free consultation.</p>
</section>

<section class="contact-section section-pad">
    <div class="contact-grid">

        <!-- Contact Information -->
        <div class="contact-info">
            <h2>Get in Touch</h2>
            <p>       <strong>Address:</strong> [Your Full Office Address, Your City]</p>
            <p>       <strong>Phone:</strong> <a href="tel:+91XXXXXXXXXX">+91-XXXXXXXXXX</a></p>
            <p>       <strong>Email:</strong>
               <a href="mailto:father@email.com">father@email.com</a></p>
            <p>       <strong>Office Hours:</strong>
    M o n Sat , 10:00 AM     6:00 PM</p>
        </div>

        <!-- Contact Form -->
        <div class="contact-form-wrapper">
            <h2>Send a Message</h2>
            <!-- action="" means "submit to the same URL (/contact)" -->
            <!-- method="post" means the data is sent as a POST request -->
            <form action="" method="post" class="contact-form">

                <div class="form-group">
                    <label for="name">Your Name *</label>
                    <!-- The 'name' attribute on input must match request.form.get('name') in app.py -->
```

```
                    <input type="text" id="name" name="name"
                           placeholder="Your full name" required>
                </div>

                <div class="form-group">
                    <label for="email">Email Address *</label>
                    <input type="email" id="email" name="email"
                           placeholder="your@email.com" required>
                </div>

                <div class="form-group">
                    <label for="message">Your Message *</label>
                    <textarea id="message" name="message" rows="5"
                              placeholder="Describe your requirement..." required></textarea>
                </div>

                <button type="submit" class="btn-primary">Send Message</button>

            </form>
        </div>

    </div>
</section>

{% endblock %}
```

Listing 2.8: templates/contact.html

2.10 Adding CSS Styling (static/css/style.css)

```
/*        style.css


   CSS makes your HTML look good.
   Without CSS, HTML is plain black text on a white page.
   CSS controls colours, fonts, spacing, and layout.

    */

/*        CSS Variables: define your colour palette once
          */
:root {
```

```
    --primary:    #003366;    /* Dark navy blue professional, trustworthy */
    --accent:     #e8a020;    /* Gold accent     authority, prestige */
    --light-bg:   #f8f9fa;    /* Very light grey for section backgrounds */
    --text:       #333333;    /* Dark grey for body text easier than pure black */
    --white:      #ffffff;
    --border:     #dddddd;
    --font-main: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

/*        Reset: remove default browser spacing        */
*, *::before, *::after {
    box-sizing: border-box;  /* Include padding/border in element width */
    margin: 0;
    padding: 0;
}

body {
    font-family: var(--font-main);
    color: var(--text);
    line-height: 1.7;        /* Space between lines improves readability */
    background: var(--white);
}

/*        Navigation Bar        */
.navbar {
    background-color: var(--primary);
    display: flex;
    justify-content: space-between;  /* Brand left, links right */
    align-items: center;
    padding: 1rem 2rem;
    position: sticky;   /* Stays at top as you scroll */
    top: 0;
    z-index: 100;       /* Appears above other content */
    box-shadow: 0 2px 8px rgba(0,0,0,0.3);
}

.nav-brand a {
    color: var(--white);
    text-decoration: none;
    font-size: 1.3rem;
    font-weight: 700;
}

```

```
.nav-links {
    list-style: none;   /* Remove bullet points from the list */
    display: flex;
    gap: 2rem;          /* Space between links */
}

.nav-links a {
    color: var(--white);
    text-decoration: none;
    font-weight: 500;
    transition: color 0.2s;  /* Smooth colour change on hover */
}

.nav-links a:hover {
    color: var(--accent);
}

/*        Hero Section        */
.hero {
    background: linear-gradient(135deg, var(--primary) 60%, #005599);
    color: var(--white);
    padding: 5rem 2rem;
    text-align: center;
}

.hero h1 {
    font-size: 2.5rem;
    margin-bottom: 1rem;
    line-height: 1.3;
}

.hero-subtitle {
    font-size: 1.1rem;
    max-width: 600px;
    margin: 0 auto 2rem;
    opacity: 0.9;
}

/*        Primary Button        */
.btn-primary {
    background: var(--accent);
    color: var(--white);
    padding: 0.8rem 2rem;
    border: none;
    border-radius: 4px;
    text-decoration: none;
    font-weight: 700;
    font-size: 1rem;
```

```
    cursor: pointer;
    display: inline-block;
    transition: background 0.2s;
}

.btn-primary:hover {
    background: #c8880e;
}

/*        Sections        */
.section-pad {
    padding: 4rem 2rem;
    max-width: 1100px;
    margin: 0 auto;   /* Centre the section on the page */
}

.section-pad h2 {
    font-size: 1.8rem;
    color: var(--primary);
    margin-bottom: 1.5rem;
    text-align: center;
}

/*        Features Grid (3 columns on desktop)        */
.features-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(250px, 1fr
   ));
    gap: 1.5rem;
    margin-top: 2rem;
}

.feature-card {
    background: var(--light-bg);
    border-left: 4px solid var(--accent);
    padding: 1.5rem;
    border-radius: 6px;
}

.feature-card h3 {
    color: var(--primary);
    margin-bottom: 0.5rem;
}

/*        Services        */
.service-card {
    border: 1px solid var(--border);
    border-radius: 8px;
    padding: 1.5rem 2rem;
    margin-bottom: 1rem;
    transition: box-shadow 0.2s;
```

```
}

.service-card:hover {
    box-shadow: 0 4px 16px rgba(0,0,0,0.1);
}

.service-card h2 {
    text-align: left;
    font-size: 1.2rem;
    margin-bottom: 0.5rem;
}

/*        Page Header        */
.page-header {
    background: var(--primary);
    color: var(--white);
    padding: 3rem 2rem;
    text-align: center;
}

.page-header h1 { font-size: 2rem; }
.page-header p  { opacity: 0.85; margin-top: 0.5rem; }

/*        Contact        */
.contact-grid {
    display: grid;
    grid-template-columns: 1fr 1.5fr;
    gap: 3rem;
}

.contact-info h2, .contact-form-wrapper h2 {
    color: var(--primary);
    margin-bottom: 1rem;
    text-align: left;
}

.contact-info p { margin-bottom: 0.8rem; }

/* Form styling */
.form-group {
    margin-bottom: 1.2rem;
}

.form-group label {
    display: block;
    font-weight: 600;
    margin-bottom: 0.3rem;
    color: var(--text);
}

.form-group input,
```

```
.form-group textarea {
    width: 100%;
    padding: 0.7rem;
    border: 1px solid var(--border);
    border-radius: 4px;
    font-size: 1rem;
    font-family: var(--font-main);
}

.form-group input:focus,
.form-group textarea:focus {
    outline: none;
    border-color: var(--primary);
    box-shadow: 0 0 0 2px rgba(0,51,102,0.15);
}

/*        Flash Messages        */
.alert {
    padding: 1rem 2rem;
    text-align: center;
    font-weight: 600;
}

.alert-success {
    background: #d4edda;
    color: #155724;
    border-bottom: 2px solid #c3e6cb;
}

/*        Footer        */
.footer {
    background: var(--primary);
    color: var(--white);
    text-align: center;
    padding: 2rem;
    margin-top: 4rem;
}

.footer p { margin-bottom: 0.4rem; opacity: 0.85; }
.footer-copy { font-size: 0.85rem; opacity: 0.6; }

/*        Mobile Responsiveness        */
/* On screens narrower than 768px (phones), stack layouts
   vertically */
@media (max-width: 768px) {
    .hero h1      { font-size: 1.7rem; }
    .navbar       { flex-direction: column; gap: 0.8rem; }
    .nav-links    { gap: 1rem; }
    .contact-grid{ grid-template-columns: 1fr; }
}
```

Listing 2.9: static/css/style.css

2.11 Running Your Website Locally

```
# 1. Open Anaconda Prompt
# 2. Activate your environment
conda activate webdev

# 3. Navigate to your project folder
cd C:\Users\YourName\Desktop\fathers_website

# 4. Run the Flask app
python app.py

# You will see output like:
# * Running on http://127.0.0.1:5000
# * Debug mode: on

# 5. Open your browser and go to:
#    http://127.0.0.1:5000
#    You should see your website's home page!
```

Listing 2.10: Run the Flask Development Server

Tip

`127.0.0.1:5000` is your *local* address — only visible on your own computer. To make it available on the internet, you need to deploy it (see next section). Press `Ctrl + C` in the terminal to stop the server.

2.12 Deploying Your Website for Free on Render

Render (https://render.com) is a free cloud platform where you can host your Flask website. Here is how.

2.12.1 Step 1: Create requirements.txt

```
# While in your project folder with webdev activated:
pip freeze > requirements.txt
# This creates a file listing all installed packages.
# Render reads this file to know what to install on its
  server.
```

Listing 2.11: Generate requirements.txt

2.12.2 Step 2: Create a Procfile

Create a file named exactly `Procfile` (no extension) in your project root:

```
web: gunicorn app:app
# This tells Render: "Start the web server using gunicorn,
# and run the 'app' object inside app.py"
```

Listing 2.12: Procfile content

2.12.3 Step 3: Push to GitHub and Connect to Render

1. Create a free account at https://github.com.
2. Create a new repository called `fathers-website`.
3. Upload all your project files to GitHub.
4. Create a free account at https://render.com.
5. Click **New Web Service** and connect your GitHub repository.
6. Render will automatically build and deploy your website.
7. You will get a free URL like `fathers-website.onrender.com`.

What You Learned

- Flask is a micro web framework: you define routes in Python that serve HTML pages.
- Templates in Jinja2 let you inherit a base layout across all pages (DRY principle).
- HTML structures the content; CSS makes it look professional.
- A contact form uses POST requests to send data to your Python function.
- `render_template` connects Python routes to HTML files.
- Render.com provides free Flask hosting connected to your GitHub repository.

Practice Task

1. Replace all `[placeholders]` in the HTML templates with your father's real name, city, qualifications, phone, and email.
2. Run `python app.py` and visit all four pages in your browser.
3. Change the `-primary` colour variable in `style.css` to a different shade of blue and see the change.
4. Create a GitHub account and push your project. Share the GitHub link

with a friend.

Chapter 3

SEO: Ranking #1 in Your City on Google

3.1 What is SEO and How Does Google Decide Rankings?

SEO stands for **Search Engine Optimisation**. It is the practice of making your website appear higher in Google's search results when someone types a query like *"tax consultant in Jamshedpur"*.

Note

Simple analogy: Imagine Google is a librarian managing millions of books. When you ask for a book on a topic, the librarian gives you the most *relevant*, *trustworthy*, and *well-organised* book first. SEO is the process of making your website the book that librarian picks first.

Google uses an algorithm with over 200 ranking factors. The main ones are:

1. **Relevance** — Does your page actually answer the search query?
2. **Authority** — Do other websites link to yours? (backlinks)
3. **User Experience** — Is the page fast, mobile-friendly, and easy to use?
4. **Local Signals** — For local searches, do you have a Google Business Profile? Are your address and phone number consistent across the web?

3.2 Keyword Research: Finding the Right Words to Target

Before writing any SEO content, you must know *which words* people actually type when searching for your father's services in your city.

3.2.1 Types of Keywords

- **Head keywords**: short, high-competition. E.g., "tax consultant". Hard to rank for because you compete with national firms.
- **Local keywords**: medium competition, high intent. E.g., "tax consultant in Jamshedpur". **This is your primary target.**
- **Long-tail keywords**: very specific, low competition, easy to rank. E.g., "income tax filing for salaried employees in Jamshedpur". Fewer searches, but people who search this are ready to hire someone.

3.2.2 Free Keyword Research Tools

1. **Google Keyword Planner** (https://ads.google.com/home/tools/keyword-planner/) — Shows search volume and competition. Requires a free Google Ads account.
2. **Google Search itself** — Type your keyword and look at the "People also ask" and "Related searches" sections at the bottom. These are real queries people use.
3. **Ubersuggest** (https://neilpatel.com/ubersuggest/) — Free (limited) keyword research tool. Shows search volume and keyword ideas.
4. **AnswerThePublic** (https://answerthepublic.com/) — Shows questions people ask around a keyword. Great for blog post ideas.

3.2.3 Your Target Keywords List

Replace "[Your City]" with your actual city name throughout this book.

- tax consultant in [Your City]
- advocate in [Your City]
- income tax filing [Your City]
- GST consultant [Your City]
- income tax return [Your City]
- tax lawyer [Your City]
- GST registration [Your City]
- best tax consultant [Your City]

3.3 On-Page SEO

On-Page SEO means optimising the content and HTML of each individual page on your website.

3.3.1 Title Tags

The title tag is the blue link text you see in Google search results. It is the single most important on-page SEO element.

Rules for title tags:

- Keep it under 60 characters (Google cuts off longer titles).
- Include your primary keyword near the beginning.
- Include your city name.
- Make it descriptive and clickable.

```
<!-- Home page -->
<title>Tax Consultant in [City] | [Father's Name]     Expert
    GST & Income Tax</title>

<!-- Services page -->
<title>Tax & Legal Services in [City] | Income Tax, GST,
    Advocacy</title>

<!-- About page -->
<title>About [Father's Name] | Advocate & Tax Consultant, [
    City]</title>

<!-- Contact page -->
<title>Contact Tax Consultant in [City] | [Father's Name]</
    title>
```

Listing 3.1: Good Title Tag Examples

3.3.2 Meta Descriptions

The meta description is the grey paragraph below the title in Google results. It does not directly affect ranking but *does* affect how many people click.

```
<!-- Home page meta description -->
<meta name="description"
      content="Looking for a trusted Tax Consultant in [City
    ]?
      [Father's Name] offers expert Income Tax filing, GST
    compliance,
      and legal advisory services. Call [Phone] for a free
    consultation.">
<!-- Keep under 155 characters -->
```

Listing 3.2: Meta Description Example

3.3.3 Heading Structure (H1–H6)

- Use **exactly one H1** per page. It should contain your primary keyword.
- Use **H2** for major sections. Include secondary keywords.
- Use **H3** for sub-sections.
- Never skip heading levels (do not jump from H1 to H4).

3.3.4 Image Alt Text

Every image on your website should have an `alt` attribute. This text tells Google (and screen readers) what the image shows.

```
<!-- Bad: no alt text -->
<img src="photo.jpg">

<!-- Bad: generic alt text -->
<img src="photo.jpg" alt="photo">

<!-- Good: descriptive, keyword-rich alt text -->
<img src="father-photo.jpg"
     alt="[Father's Name], Tax Consultant and Advocate in [Your City]">
```

Listing 3.3: Image Alt Text Example

3.3.5 Keyword Placement Checklist

For each page, your target keyword should appear in:

☐ The `<title>` tag

☐ The meta description

☐ The H1 heading

☐ At least one H2 heading

☐ The first paragraph of body text

☐ At least 2–3 times in the body content (naturally, not forced)

☐ Image alt text

☐ The page URL (e.g., `/tax-consultant-jamshedpur`)

3.4 Technical SEO

Technical SEO ensures Google can find, crawl, and understand your website.

3.4.1 Creating sitemap.xml

A sitemap is a file that lists all the pages on your website. It helps Google find and index your pages faster. In Chapter 5, we will write a Python script to generate this automatically. For now, create `static/sitemap.xml` manually:

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">

  <url>
    <loc>https://www.yourwebsite.com/</loc>
    <changefreq>monthly</changefreq>
    <priority>1.0</priority>
    <!-- priority 1.0 = most important page (home) -->
  </url>

  <url>
    <loc>https://www.yourwebsite.com/about</loc>
    <changefreq>yearly</changefreq>
    <priority>0.7</priority>
  </url>

  <url>
    <loc>https://www.yourwebsite.com/services</loc>
    <changefreq>monthly</changefreq>
    <priority>0.9</priority>
  </url>

  <url>
    <loc>https://www.yourwebsite.com/contact</loc>
    <changefreq>yearly</changefreq>
    <priority>0.6</priority>
  </url>

</urlset>
```

Listing 3.4: sitemap.xml

3.4.2 Creating robots.txt

`robots.txt` tells Google which pages to crawl and which to ignore.

```
User-agent: *
Allow: /

Sitemap: https://www.yourwebsite.com/sitemap.xml
```

Listing 3.5: static/robots.txt

Serve these files from Flask by adding routes to `app.py`:

```
from flask import send_from_directory  # Add this import at
    the top

```

```python
@app.route('/sitemap.xml')
def sitemap():
    """Serve the sitemap.xml file to search engines."""
    return send_from_directory('static', 'sitemap.xml')

@app.route('/robots.txt')
def robots():
    """Serve robots.txt to search engine crawlers."""
    return send_from_directory('static', 'robots.txt')
```

Listing 3.6: Add sitemap and robots.txt routes to app.py

3.4.3 Page Speed Optimisation

Page speed is a Google ranking factor. Faster pages rank higher.

- **Compress images** before uploading. Use https://squoosh.app (free). Aim for images under 100KB.
- **Minimise CSS**: remove unused styles.
- **Use Google's PageSpeed Insights** (https://pagespeed.web.dev/) to test your page and get specific recommendations.

3.5 Local SEO: Dominating Your City

Local SEO is the most powerful tool for your father's business. When someone searches "tax consultant near me" or "tax consultant in [City]", Google shows a **Local Pack** — a map with 3 local businesses. Getting into that Local Pack can bring your father more clients than anything else.

3.5.1 Google Business Profile (GBP)

Google Business Profile (formerly Google My Business) is a **free** tool that puts your father's practice on Google Maps and in the Local Pack.

Steps to set up:

1. Go to https://business.google.com and sign in with a Google account.
2. Click **Add your business to Google**.
3. Enter the business name exactly as it should appear everywhere.
4. Select the business category: **Tax Consultant** and/or **Lawyer**.
5. Enter the full address. This must be a real, physical address.
6. Enter the phone number. **Use the same number on the website.**
7. Enter the website URL.
8. Google will send a postcard with a verification code to the address. Enter the code to verify. This usually takes 5–14 days.

3.5.2 NAP Consistency

NAP stands for **Name, Address, Phone Number**. These three pieces of information must be *100% identical* everywhere online: your website, Google Business Profile, Facebook, JustDial, Sulekha, and any other directory.

Warning

Even tiny differences hurt your rankings. "St." vs "Street", or "+91" vs "91", or "Road" vs "Rd" — Google treats these as different businesses and gets confused. Pick one format and use it everywhere.

3.5.3 Getting Google Reviews

Reviews are one of the most powerful local SEO signals.

How to get reviews:

1. From your Google Business Profile dashboard, find the direct review link.
2. Share this link with satisfied clients via WhatsApp or email.
3. Ask them to write a 3–5 sentence review mentioning the service they used and the city (e.g., "Best tax consultant in [City] for GST filing!").
4. Aim for at least 10–20 genuine reviews to start.
5. **Always respond** to every review — both positive and negative. This shows Google and potential clients that you are active and professional.

3.5.4 Local Citations

A **citation** is any mention of your father's business name, address, and phone number on another website. Even without a link, citations help local SEO.

Indian directories to submit to (free):

- JustDial (https://www.justdial.com)
- Sulekha (https://www.sulekha.com)
- IndiaMART (https://www.indiamart.com)
- TradeIndia (https://www.tradeindia.com)
- Practo (for advisory/consultation services)
- Local chamber of commerce directory in your city

3.6 Schema Markup for Rich Results

Schema markup is a special type of code you add to your HTML that helps Google understand your content more precisely. It can make your result in Google show extra information like your address, phone number, and reviews directly in the search results (called **rich results**).

```
<!-- Add this inside the <head> tag of your base.html -->
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "LegalService",
  "name": "[Father's Full Name]",
  "description": "Tax Consultant and Advocate in [Your City] specialising
                  in Income Tax, GST, and Legal Advisory.",
  "url": "https://www.yourwebsite.com",
  "telephone": "+91-XXXXXXXXXX",
  "address": {
    "@type": "PostalAddress",
    "streetAddress": "[Street Address]",
    "addressLocality": "[Your City]",
    "addressRegion": "[Your State]",
    "postalCode": "[PIN Code]",
    "addressCountry": "IN"
  },
  "openingHours": "Mo-Sa 10:00-18:00",
  "priceRange": "$$",
  "areaServed": "[Your City]"
}
</script>
```

Listing 3.7: Schema Markup for a Local Business

3.7 Setting Up Google Search Console

Google Search Console (GSC) is a free tool from Google that shows you:

- Which keywords people use to find your website
- How many people click on your website in search results
- Which pages are indexed (and which are not)
- Any technical errors Google found on your site

Setup steps:

1. Go to https://search.google.com/search-console/
2. Click **Add Property** and enter your website URL.

3. Verify ownership by adding a meta tag to your `base.html`:

```
<meta name="google-site-verification" content="
    YOUR_CODE_HERE" />

```

4. Submit your sitemap: in GSC, go to **Sitemaps** and enter `https://www.yourwebsite.com/sitema`

5. Wait 48–72 hours for Google to verify and start showing data.

3.8 Complete SEO Checklist

SEO Checklist

On-Page SEO:

- ☐ Unique title tag on every page (under 60 chars, keyword-first)
- ☐ Unique meta description on every page (under 155 chars)
- ☐ One H1 per page with primary keyword
- ☐ H2/H3 structure used logically
- ☐ Alt text on all images
- ☐ Keyword appears naturally in first paragraph
- ☐ NAP in footer of every page

Technical SEO:

- ☐ sitemap.xml created and submitted to GSC
- ☐ robots.txt created and accessible
- ☐ Website loads in under 3 seconds (test at pagespeed.web.dev)
- ☐ Website is mobile-responsive (test at search.google.com/test/mobile-friendly)
- ☐ HTTPS enabled (Render provides this automatically)
- ☐ Schema markup added to home page

Local SEO:

- ☐ Google Business Profile created and verified
- ☐ NAP consistent on website, GBP, and all directories
- ☐ At least 10 genuine Google reviews
- ☐ Listed on JustDial, Sulekha, IndiaMART
- ☐ Google Search Console set up with sitemap submitted

What You Learned

- SEO makes your website appear in Google search results for relevant queries.
- Keyword research identifies the exact phrases people search for in your city.
- On-page SEO optimises title tags, meta descriptions, headings, and content.
- Technical SEO ensures Google can crawl and index your site efficiently.

- Local SEO — especially Google Business Profile and NAP consistency — is the fastest way to rank for local searches.
- Schema markup gives Google structured data about your business.
- Google Search Console monitors your search performance for free.

Practice Task

1. Update the title tags and meta descriptions in all four templates using the rules from this chapter.
2. Add the Schema markup to `base.html` with your father's real details.
3. Create a free Google Business Profile for your father's practice.
4. Run your website through https://pagespeed.web.dev and note the score.
5. List at least 3 long-tail keywords no one else would easily think of for your father's specific services.

Chapter 4

Managing Your Father's LinkedIn Profile

4.1 Why LinkedIn Matters for a Tax Consultant

LinkedIn is the world's largest professional network with over 900 million members. For a Tax Consultant and Advocate, LinkedIn serves three purposes:

1. **Credibility**: A well-optimised LinkedIn profile appears in Google search results when someone searches your father's name.

2. **Referrals**: Other professionals (CAs, lawyers, business owners) can discover your father and refer clients.

3. **Content**: By publishing useful tax tips and legal insights, your father builds authority as a trusted expert in his city.

4.2 Optimising the LinkedIn Profile

4.2.1 Profile Photo and Banner

- **Profile photo**: A professional headshot against a plain background. Wear formal attire. Good lighting. This is non-negotiable.

- **Banner image**: A custom banner (1584 x 396 px) showing your city name, services offered, and contact number. Create it free on Canva (https://canva.com).

4.2.2 Headline

The headline is the line that appears under your name. It is indexed by LinkedIn's search algorithm. Do not just write "Advocate" — that is too generic.

Formula: *[Primary Role] | [Key Service 1] | [Key Service 2] | [City]*

Note

Example headline:
Tax Consultant & Advocate | Income Tax · GST · Legal Advisory | [City], India

4.2.3 About Section (Summary)

The About section is your father's professional story. Write 200–300 words. Include:

- Years of experience
- Specific services and who they help (target clients)
- Notable achievements (number of clients served, cases won, etc.)
- City and region he serves
- A call to action (e.g., "Connect with me for a free initial consultation")
- Keywords: income tax, GST, tax consultant, advocate, [Your City]

4.2.4 Experience Section

- List his current practice as a company (create a LinkedIn Company Page for it).
- Write 3–5 bullet points describing his role and achievements.
- Include keywords in each bullet point.

4.2.5 Skills Section

Add at least 20 relevant skills. Prioritise: *Income Tax, GST, Tax Planning, Tax Advisory, Legal Research, Corporate Law, Income Tax Appeals, Financial Planning, Business Registration, Tax Compliance, TDS, Bookkeeping, [Your City], India.*

Ask colleagues and clients to endorse these skills.

4.2.6 Featured Section

The Featured section appears near the top of the profile. Add your father's website link here, and any useful articles he writes on LinkedIn.

4.3 LinkedIn SEO: Ranking in LinkedIn Search

People search LinkedIn for professionals just like they search Google. To rank for "tax consultant [City]" in LinkedIn's search:

- Include your city name in the headline, about section, and location field.
- Repeat your primary keyword ("tax consultant", "advocate") naturally in the headline, about section, and experience descriptions.

- Complete **100% of the profile**. LinkedIn boosts complete profiles in its search results. Make sure all sections are filled.
- Get **connections**: the more 1st-degree connections, the higher you rank. Connect with every colleague, client, and local professional you know.

4.4 Content Strategy and Calendar

Consistency is the key to LinkedIn growth. Here is a simple, sustainable content plan.

4.4.1 Post Frequency

Target: 3 posts per week. This is achievable and more effective than posting every day for one week and then disappearing.

4.4.2 Content Types (with Examples)

1. **Quick Tax Tips (2 posts/week):** Short, useful information. Example:

   > "Did you know? Under Section 80D, you can claim up to 25,000 deduction for health insurance premiums. If your parents are senior citizens, this goes up to 50,000. Are you claiming it? #IncomeTax #TaxTips #[City]"

2. **Deadline Reminders (1 post/week during tax season):** Example: "ITR filing deadline is July 31. Don't wait for the last day — contact us at [Phone] to get started."
3. **Client Success Stories (2x/month, with permission):** "We helped a small business owner in [City] save [X] through proper tax planning. Here is how." (keep it anonymous or with permission)
4. **Short Articles (1/month):** Write a 500-word article on LinkedIn about a topic like "GST Registration: A Complete Guide for Small Businesses in [City]". These rank in Google search results.
5. **Seasonal Content:** ITR season (July), Advance Tax deadlines (March, June, Sept, Dec), Budget commentary (February).

4.4.3 Monthly Content Calendar Template

Week	**Monday**	**Wednesday**	**Friday**
Week 1	Quick Tax Tip	Deadline Reminder	Quick Tax Tip
Week 2	Client Story	GST Update	Quick Tax Tip
Week 3	Quick Tax Tip	Article (Long form)	Seasonal Content
Week 4	Quick Tax Tip	Q&A / Myth-bust	Month Recap

Table 4.1: Monthly LinkedIn Content Calendar

4.5 Growing the Professional Network

1. **Connect with local professionals:** CAs, lawyers, business owners, finance managers in your city. Send personalised connection requests.
2. **Join LinkedIn Groups:** Search for "Tax Professionals India", "GST India", "Income Tax Practitioners". Engage genuinely in discussions.
3. **Engage with others:** Comment thoughtfully on posts by local professionals. This increases profile visibility.
4. **Tag your city:** Use #[YourCity] and #[YourState] in every post to appear in local searches.

4.6 Metrics to Track

- **Profile views:** Check weekly. Should grow month over month.
- **Search appearances:** Shown in LinkedIn analytics. Shows how many people found the profile in search.
- **Post impressions:** How many people saw each post.
- **Post engagement rate:** (Likes + Comments + Shares) / Impressions. A rate above 2% is good for LinkedIn.
- **Connection count:** Aim for 500+ connections within 6 months.

What You Learned

- A complete LinkedIn profile with city-specific keywords ranks in both LinkedIn and Google searches.
- The headline formula: Role | Service 1 | Service 2 | City.
- Posting 3 times per week with useful, relevant content builds authority over time.
- Engaging with local professionals amplifies visibility without paid advertising.
- Track profile views, search appearances, and post engagement monthly.

Practice Task

1. Create (or update) your father's LinkedIn profile using all the sections described.
2. Write a headline using the formula from this chapter.
3. Write the About section (200–300 words) based on the structure given.
4. Draft the first 4 LinkedIn posts using the content calendar template.

5. Make a list of 20 local professionals to send connection requests to.

Chapter 5

Python Automation for SEO and Web Management

5.1 Introduction: Why Automate?

Manual SEO tasks — checking your keyword rankings, auditing your pages, monitoring your website — take time. Python lets you automate these tasks so they happen on a schedule without you doing anything. This chapter gives you five real, working scripts with full explanations.

Note

Folder structure for this chapter: Create a folder called `seo_automation` inside your `fathers_website` folder. Put all the scripts from this chapter there.

5.2 Script 1: Keyword Ranking Tracker

This script checks where your website ranks on Google for a list of keywords and saves the results to an Excel file so you can track progress over time.

Warning

Important: Google blocks automated searches. This script uses the `googlesearch-python` library which works for personal/educational use at low frequency. Do not run it more than once per day. For production use, consider the Google Search Console API (Chapter 6 covers resources for this).

```
pip install googlesearch-python
```

Listing 5.1: Install googlesearch-python

```
#         keyword_tracker.py


# This script checks the Google search ranking position of
    your
```

```python
# website for a list of keywords and saves the results to Excel.

import pandas as pd                     # For storing and saving data
from googlesearch import search         # For searching Google
from datetime import datetime           # For recording the date of each check
import time                             # For adding delays between searches
import os                               # For file path operations

#         CONFIGURATION


# Edit these two variables to match your website and city.
YOUR_WEBSITE = "yourwebsite.com"        # Part of your URL to look for in results
YOUR_CITY    = "Jamshedpur"             # Replace with your actual city

# List of keywords to track. Add or remove as needed.
KEYWORDS = [
    f"tax consultant in {YOUR_CITY}",
    f"advocate in {YOUR_CITY}",
    f"income tax filing {YOUR_CITY}",
    f"GST consultant {YOUR_CITY}",
    f"income tax return {YOUR_CITY}",
    f"best tax consultant {YOUR_CITY}",
]

# How many Google results to check (check top 30 positions)
RESULTS_TO_CHECK = 30

# Where to save the Excel file (in the same folder as this script)
OUTPUT_FILE = "keyword_rankings.xlsx"

#         FUNCTION: Check ranking for one keyword

def check_ranking(keyword, website, num_results=30):
    """
    Searches Google for a keyword and checks what position
    the given website appears at.

    Returns the ranking position (1-30), or 'Not in Top 30' if not found.
    """
    try:
        # search() returns a generator of URLs from Google
```

```
    results
        # list() converts it to a Python list
        results = list(search(keyword, num_results=num_results))

        # Loop through results and check if our website is in the URL
        for position, url in enumerate(results, start=1):
            # enumerate starts counting from 1 (position 1 = top result)
            if website.lower() in url.lower():
                return position  # Found! Return its position number

        return f"Not in Top {num_results}"  # Not found in any result

    except Exception as e:
        # If anything goes wrong (network error, block, etc.), report it
        return f"Error: {str(e)}"

#         MAIN SCRIPT


def main():
    print("=" * 50)
    print("Keyword Ranking Tracker")
    print(f"Website: {YOUR_WEBSITE}")
    print(f"Date: {datetime.now().strftime('%Y-%m-%d %H:%M')}")
    print("=" * 50)

    # Store results in a list of dictionaries
    results_data = []

    for keyword in KEYWORDS:
        print(f"Checking: '{keyword}'...", end=" ")

        ranking = check_ranking(keyword, YOUR_WEBSITE, RESULTS_TO_CHECK)

        print(f"Position: {ranking}")

        # Add this result to our data list
        results_data.append({
            "Date"     : datetime.now().strftime("%Y-%m-%d"),
            "Keyword"  : keyword,
            "Position": ranking,
            "Website"  : YOUR_WEBSITE
        })
```

```python

        # Wait 3 seconds between searches to avoid being blocked by Google
        time.sleep(3)

    # Convert results to a pandas DataFrame (like a table)
    df_new = pd.DataFrame(results_data)

    # If the Excel file already exists, load it and append new results.
    # This builds up a history of rankings over time.
    if os.path.exists(OUTPUT_FILE):
        df_existing = pd.read_excel(OUTPUT_FILE)
        df_combined = pd.concat([df_existing, df_new], ignore_index=True)
    else:
        # First run: start fresh
        df_combined = df_new

    # Save the combined data back to Excel
    df_combined.to_excel(OUTPUT_FILE, index=False)
    # index=False means don't add a row number column

    print(f"\ n   Results saved to: {OUTPUT_FILE}")
    print("Run this script weekly to track your progress over time.")

# Python convention: only run main() if this script is run directly,
# not if it is imported by another script.
if __name__ == "__main__":
    main()
```

Listing 5.2: seo_automation/keyword_tracker.py

5.3 Script 2: On-Page SEO Auditor

This script visits your website pages and checks whether each one has proper title tags, meta descriptions, H1 headings, and working links.

```python
#         seo_auditor.py


# Audits the on-page SEO of each page of your website.
# Checks: title, meta description, H1 tags, broken links.
# Prints a report and saves it to a CSV file.

import requests                          # To fetch web pages
from bs4 import BeautifulSoup            # To parse the HTML of each page
```

```python
from urllib.parse import urljoin, urlparse  # For handling URLs
import pandas as pd                         # To save the report
from datetime import datetime

#         CONFIGURATION


BASE_URL = "https://yourwebsite.com"   # Replace with your real website URL

# Pages to audit. Add more as your website grows.
PAGES_TO_AUDIT = [
    "/",            # Home
    "/about",       # About
    "/services",    # Services
    "/contact",     # Contact
]

OUTPUT_FILE = "seo_audit_report.csv"

#         FUNCTION: Audit a single page


def audit_page(url):
    """
    Fetches a single URL and checks its SEO elements.
    Returns a dictionary of findings.
    """
    result = {
        "URL"               : url,
        "Status Code"       : None,     # HTTP status (200=OK, 404=Not Found)
        "Title"             : None,     # The <title> tag content
        "Title Length"      : None,     # Should be 10-60 characters
        "Meta Description" : None,     # The meta description content
        "Meta Desc Length" : None,     # Should be 50-155 characters
        "H1 Count"          : None,     # Should be exactly 1
        "H1 Text"           : None,     # What the H1 says
        "Broken Links"      : [],       # List of broken links on the page
        "Issues"            : []        # List of SEO issues found
    }

    try:
        # Fetch the page using requests
```

```
        response = requests.get(url, timeout=10)
        # timeout=10 means give up if the server takes more than 10 seconds

        result["Status Code"] = response.status_code

        if response.status_code != 200:
            result["Issues"].append(f"Page returned status {response.status_code}")
            return result

        # Parse the HTML with BeautifulSoup
        # 'html.parser' is Python's built-in HTML parser
        soup = BeautifulSoup(response.text, 'html.parser')

        #         Check Title Tag


        title_tag = soup.find('title')          # Find the <title> element
        if title_tag and title_tag.string:
            title_text = title_tag.string.strip()
            result["Title"]         = title_text
            result["Title Length"] = len(title_text)

            if len(title_text) < 10:
                result["Issues"].append("Title too short (under 10 chars)")
            elif len(title_text) > 60:
                result["Issues"].append(f"Title too long ({len(title_text)} chars, aim for under 60)")
        else:
            result["Issues"].append("MISSING: No <title> tag found!")

        #         Check Meta Description


        meta_desc = soup.find('meta', attrs={'name': 'description'})
        # soup.find() searches the HTML for a specific tag with specific attributes

        if meta_desc and meta_desc.get('content'):
            desc_text = meta_desc['content'].strip()
            result["Meta Description"] = desc_text
            result["Meta Desc Length"] = len(desc_text)

            if len(desc_text) < 50:
                result["Issues"].append("Meta description too short (under 50 chars)")
```

```
            elif len(desc_text) > 155:
                result["Issues"].append(f"Meta description too long ({len(desc_text)} chars)")
        else:
            result["Issues"].append("MISSING: No meta description found!")

        #         Check H1 Tags


        h1_tags = soup.find_all('h1')           # Find ALL <h1> elements
        result["H1 Count"] = len(h1_tags)

        if len(h1_tags) == 0:
            result["Issues"].append("MISSING: No H1 heading found!")
        elif len(h1_tags) > 1:
            result["Issues"].append(f"Multiple H1 tags found ({len(h1_tags)}). Use exactly one H1 per page.")
        else:
            result["H1 Text"] = h1_tags[0].get_text(strip=True)

        #         Check for Broken Links


        all_links = soup.find_all('a', href=True)
        # soup.find_all() returns a list of ALL <a> tags that have an href attribute

        for link in all_links:
            href = link['href']

            # Skip email links, phone links, and anchor links
            if href.startswith(('mailto:', 'tel:', '#', 'javascript:')):
                continue

            # Build the full URL (handles relative URLs like /about)
            full_url = urljoin(url, href)

            # Only check links on the same domain (internal links)
            if urlparse(full_url).netloc == urlparse(url).netloc:
                try:
                    link_response = requests.head(full_url, timeout=5)
                    # requests.head() fetches only the
```

```
    headers (faster than full page)

                    if link_response.status_code == 404:
                        result["Broken Links"].append(full_url)
                        result["Issues"].append(f"Broken link: {full_url}")
                except:
                    pass  # Ignore network errors for link checks

    except requests.exceptions.ConnectionError:
        result["Issues"].append("Could not connect to the website. Is it running?")
    except Exception as e:
        result["Issues"].append(f"Error during audit: {str(e)}")

    return result

#        MAIN SCRIPT


def main():
    print("=" * 60)
    print("SEO Audit Report")
    print(f"Website : {BASE_URL}")
    print(f"Date    : {datetime.now().strftime('%Y-%m-%d %H:%M')}")
    print("=" * 60)

    all_results = []

    for page_path in PAGES_TO_AUDIT:
        url = BASE_URL + page_path
        print(f"\nAuditing: {url}")

        result = audit_page(url)

        # Print summary to the console
        print(f"  Status       : {result['Status Code']}")
        print(f"  Title        : {result['Title']} ({result['Title Length']} chars)")
        print(f"  Meta Desc    : {result['Meta Desc Length']} chars")
        print(f"  H1 Count     : {result['H1 Count']}")

        if result["Issues"]:
            print(f"          Issues ({len(result['Issues'])}):")
            for issue in result["Issues"]:
```

```
                print(f"      - {issue}")
        else:
            print("      No issues found!")

        all_results.append(result)

    # Save to CSV
    df = pd.DataFrame(all_results)
    # Convert lists to strings for CSV storage
    df["Issues"]       = df["Issues"].apply(lambda x: "; ".join(x) if x else "None")
    df["Broken Links"] = df["Broken Links"].apply(lambda x: "; ".join(x) if x else "None")
    df.to_csv(OUTPUT_FILE, index=False)

    print(f"\ n   Audit report saved to: {OUTPUT_FILE}")

if __name__ == "__main__":
    main()
```

Listing 5.3: seo_automation/seo_auditor.py

5.4 Script 3: Sitemap Generator

This script automatically generates an up-to-date `sitemap.xml` for your website every time you add a new page.

```
#         generate_sitemap.py


# Generates a sitemap.xml file for your Flask website.
# Run this script whenever you add a new page to your website.

from datetime import datetime
import os

#         CONFIGURATION


BASE_URL   = "https://yourwebsite.com"  # Replace with your website URL
OUTPUT_DIR = "../static"                # Save sitemap to static/ folder
# "../" means "go up one folder" from seo_automation/ to fathers_website/

# List of pages with their SEO settings
# changefreq: how often Google should re-check this page
# priority  : how important is this page (1.0 = most important, 0.1 = least)
```

```
PAGES = [
    {"path": "/",          "changefreq": "monthly", "priority": "1.0"},
    {"path": "/about",     "changefreq": "yearly",  "priority": "0.7"},
    {"path": "/services",  "changefreq": "monthly", "priority": "0.9"},
    {"path": "/contact",   "changefreq": "yearly",  "priority": "0.6"},
]

#         BUILD THE XML


def generate_sitemap():
    """Generates the sitemap.xml content as a string."""

    today = datetime.now().strftime("%Y-%m-%d")  # e.g., "2025-03-15"

    # Start with the XML header and opening tag
    xml_lines = [
        '<?xml version="1.0" encoding="UTF-8"?>',
        '<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">'
    ]

    # Add one <url> block for each page
    for page in PAGES:
        full_url = BASE_URL + page["path"]
        xml_lines.append("  <url>")
        xml_lines.append(f"    <loc>{full_url}</loc>")
        xml_lines.append(f"    <lastmod>{today}</lastmod>")
        xml_lines.append(f"    <changefreq>{page['changefreq']}</changefreq>")
        xml_lines.append(f"    <priority>{page['priority']}</priority>")
        xml_lines.append("  </url>")

    # Close the root tag
    xml_lines.append("</urlset>")

    # Join all lines with newline characters into one string
    return "\n".join(xml_lines)

def main():
    sitemap_content = generate_sitemap()

    # Build the full path to save the file
    output_path = os.path.join(OUTPUT_DIR, "sitemap.xml")

```

```
    # Write the sitemap content to the file
    with open(output_path, "w", encoding="utf-8") as f:
        f.write(sitemap_content)
        # "w" means write mode (creates the file if it doesn't exist)
        # encoding="utf-8" ensures special characters are handled correctly

    print(f"    Sitemap generated successfully: {output_path}")
    print(f"   Pages included: {len(PAGES)}")
    print(f"   Last modified date: {datetime.now().strftime('%Y-%m-%d')}")
    print()
    print("Next step: Submit your sitemap to Google Search Console.")
    print(f"Sitemap URL: {BASE_URL}/sitemap.xml")

if __name__ == "__main__":
    main()
```

Listing 5.4: seo_automation/generate_sitemap.py

5.5 Script 4: Website Uptime Monitor

This script checks if your website is online every 30 minutes and prints an alert if it goes down.

```
#         uptime_monitor.py


# Checks if your website is online every 30 minutes.
# Prints a warning if the website is down.
# (For real alerting via SMS/email, you'd extend this with an email library)

import requests        # To check the website
import schedule        # To run the check on a schedule
import time            # For the main loop
from datetime import datetime

#         CONFIGURATION


WEBSITE_URL      = "https://yourwebsite.com"     # Replace with your URL
CHECK_INTERVAL   = 30                             # Check every 30 minutes
LOG_FILE         = "uptime_log.txt"              # Log file to record all checks

```

```python
#         FUNCTION: Check if website is up


def check_website():
    """
    Tries to connect to your website.
    If it responds with HTTP 200 (OK), the site is up.
    Anything else, or a connection error, means it is down.
    """
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

    try:
        # Send a GET request to the website
        # timeout=10 : if no response in 10 seconds, assume it is down
        response = requests.get(WEBSITE_URL, timeout=10)

        if response.status_code == 200:
            # status_code 200 means "OK"     the page loaded successfully
            status = "    UP"
            message = f"[{timestamp}] {status} | {WEBSITE_URL} | Response: {response.status_code}"
        else:
            # Any other status code (like 500, 503) means something is wrong
            status = "       ISSUE"
            message = (f"[{timestamp}] {status} | {WEBSITE_URL} | "
                       f"Unexpected status: {response.status_code}")
            print(f"\  n     ALERT: Website returned status {response.status_code}!")

    except requests.exceptions.ConnectionError:
        # ConnectionError means the website is completely unreachable
        status = "    DOWN"
        message = f"[{timestamp}] {status} | {WEBSITE_URL} | CONNECTION REFUSED"
        print(f"\ n   ALERT: Your website appears to be DOWN!")
        print(f"   Time : {timestamp}")
        print(f"   URL  : {WEBSITE_URL}")
        print(f"   Action: Check your hosting (Render dashboard)")

    except requests.exceptions.Timeout:
        status = "    TIMEOUT"
        message = f"[{timestamp}] {status} | {WEBSITE_URL} | Request timed out"
```

```
        print(f"\  n      ALERT: Website is not responding (timeout)!")

    # Print to console
    print(message)

    # Append to log file
    with open(LOG_FILE, "a") as f:     # "a" = append mode (doesn't overwrite)
        f.write(message + "\n")

#         SCHEDULE THE CHECK


# Run check_website() every 30 minutes
schedule.every(CHECK_INTERVAL).minutes.do(check_website)

#         MAIN LOOP


def main():
    print(f"        Uptime Monitor started for: {WEBSITE_URL}")
    print(f"    Checking every {CHECK_INTERVAL} minutes.")
    print(f"    Press Ctrl+C to stop.\n")

    # Run the first check immediately when the script starts
    check_website()

    # Keep running forever (until you press Ctrl+C)
    while True:
        # schedule.run_pending() checks if any scheduled task is due
        schedule.run_pending()
        time.sleep(60)     # Check every 60 seconds whether a scheduled task is due

if __name__ == "__main__":
    main()
```

Listing 5.5: seo_automation/uptime_monitor.py

5.6 Script 5: Running Scripts on a Schedule (Windows)

On Windows, the best way to schedule Python scripts to run automatically is using **Windows Task Scheduler**.

```
REM Create a file called run_keyword_tracker.bat with this content:
```

```
REM (Right-click the desktop -> New -> Text Document -> rename to .bat)

@echo off
REM Activate the conda environment
call C:\Users\YourName\anaconda3\Scripts\activate.bat webdev

REM Run the keyword tracker script
python C:\Users\YourName\Desktop\fathers_website\seo_automation\keyword_tracker.py

REM Pause so you can see the output (remove this for silent running)
pause
```

Listing 5.6: Create a .bat launcher file

Then in Windows Task Scheduler:

1. Open "Task Scheduler" from the Start Menu.
2. Click **Create Basic Task**.
3. Name it "Keyword Tracker — Weekly".
4. Set trigger to **Weekly** on a day of your choice.
5. Set action to **Start a Program** and browse to your `.bat` file.
6. Click Finish.

What You Learned

- Python scripts can automate repetitive SEO tasks: tracking rankings, auditing pages, generating sitemaps, and monitoring uptime.
- `requests` fetches web pages; `BeautifulSoup` parses their HTML.
- `pandas` stores structured data in DataFrames and saves to Excel/CSV.
- The `schedule` library runs functions at set time intervals.
- Windows Task Scheduler can run `.bat` files on a weekly schedule without any manual effort.

Practice Task

1. Run `seo_auditor.py` against your local Flask website (while it is running on localhost).
2. Read the CSV output and identify at least one real issue to fix in your HTML templates.
3. Run `generate_sitemap.py` and verify that `sitemap.xml` was updated in the `static/` folder.

4. Set up the `.bat` file and test it by double-clicking it.

Chapter 6

12-Week Learning and Execution Roadmap

6.1 How to Use This Roadmap

This roadmap is your study and build plan for the next 12 weeks. Each week has four parts:

1. **Topics to Study** — concepts to understand.
2. **Practical Task** — something real to build or do.
3. **Resources** — where to learn more (all free).
4. **Expected Outcome** — what you should have by end of week.

Warning

Important: Do not rush. Each week is designed to take 5–10 hours total. If you spend 1 hour per weekday and 2–3 hours on weekends, you will finish on time. The most important thing is consistency — doing a little every day beats doing a lot in one sitting.

6.2 Week 1 – Anaconda, Python, and Your First Flask Page

- **Study:** Chapter 1 of this book. Anaconda setup, virtual environments, pip vs conda. Read the official Flask quickstart: https://flask.palletsprojects.com/quickstart/.
- **Task:** Set up the `webdev` environment. Install all packages. Run Flask's "Hello World" example. Show it in your browser.
- **Resources:** Official Flask docs (https://flask.palletsprojects.com), CS50x on edX (free, covers web fundamentals).
- **Outcome:** A working Flask app that displays "Hello, Minza!" in the browser.

6.3 Week 2 – HTML and CSS Fundamentals

- **Study:** What are HTML tags? Block vs inline elements. CSS box model. CSS selectors (class, id, tag). Flexbox and Grid basics.
- **Task:** Create the `base.html` template and `style.css` from Chapter 2. View them in your browser. Experiment with colour changes.
- **Resources:** MDN Web Docs HTML (https://developer.mozilla.org/en-US/docs/Web/HTML), CSS-Tricks Flexbox guide (https://css-tricks.com/snippets/css/a-guide-to-flexbox/).
- **Outcome:** A styled navigation bar and footer that appear on all pages.

6.4 Week 3 – Flask Templates and All Four Pages

- **Study:** Jinja2 template inheritance (`{% extends %}`, `{% block %}`). Flask `url_for()`. Static files.
- **Task:** Build all four pages (Home, About, Services, Contact) with real content about your father. Replace all placeholders with real information.
- **Resources:** Jinja2 docs (https://jinja.palletsprojects.com), Corey Schafer's Flask tutorial series on YouTube (search: "Corey Schafer Flask").
- **Outcome:** A complete 4-page website running locally on your laptop.

6.5 Week 4 – Deploying to the Internet

- **Study:** What is Git and GitHub? Basic commands: `git init`, `git add`, `git commit`, `git push`. What is a web server? What is gunicorn?
- **Task:** Create a GitHub account. Push your project. Deploy on Render.com. Share the live URL with your family.
- **Resources:** GitHub docs (https://docs.github.com/en/get-started), Render deployment guide (https://render.com/docs/deploy-flask).
- **Outcome:** Your father's website is live on the internet at a real URL!

6.6 Week 5 – On-Page SEO Implementation

- **Study:** Chapter 3 Sections 1–4. Title tags, meta descriptions, H1/H2 rules, image alt text, keyword placement.
- **Task:** Update every page with proper SEO titles, descriptions, and headings. Add schema markup to `base.html`. Run Lighthouse audit in Chrome (F12 → Lighthouse tab).

- **Resources:** Google's SEO Starter Guide (https://developers.google.com/search/docs/fundamentals/seo-starter-guide), Moz Beginner's Guide to SEO (https://moz.com/beginners-guide-to-seo).
- **Outcome:** Every page has a unique, keyword-rich title and meta description. Lighthouse SEO score above 90.

6.7 Week 6 – Technical SEO and Google Search Console

- **Study:** Sitemap.xml, robots.txt, page speed, mobile-friendliness, HTTPS.
- **Task:** Run Script 3 to generate sitemap. Create robots.txt. Set up Google Search Console. Submit sitemap. Test mobile-friendliness.
- **Resources:** Google Search Console Help (https://support.google.com/webmasters), PageSpeed Insights (https://pagespeed.web.dev).
- **Outcome:** Google Search Console is set up. Sitemap is submitted. Google begins indexing your pages.

6.8 Week 7 – Local SEO: Google Business Profile

- **Study:** Local SEO signals. NAP consistency. Google Business Profile. Indian local directories.
- **Task:** Create and completely fill out the Google Business Profile. Submit NAP to JustDial, Sulekha, and IndiaMART. Ask 2–3 satisfied clients to leave a Google review.
- **Resources:** Google Business Profile Help (https://support.google.com/business), BrightLocal's Local SEO guide (https://www.brightlocal.com/learn/local-seo/).
- **Outcome:** Google Business Profile is created and verification postcard is requested.

6.9 Week 8 – LinkedIn Profile Setup

- **Study:** Chapter 4. LinkedIn profile anatomy. LinkedIn SEO. Profile completeness.
- **Task:** Build the complete LinkedIn profile for your father using the structure from Chapter 4. Connect with 20 local professionals.
- **Resources:** LinkedIn Learning (free with LinkedIn account) — search "Optimise your LinkedIn profile".
- **Outcome:** A 100% complete LinkedIn profile. First 20 connections made.

6.10 Week 9 – Python SEO Automation: Auditor and Sitemap

- **Study:** Chapter 5. `requests` library. `BeautifulSoup` basics. Parsing HTML. Reading and writing files in Python.
- **Task:** Run Script 2 (SEO Auditor) on your live website. Fix the issues it finds. Run Script 3 (Sitemap Generator).
- **Resources:** BeautifulSoup documentation (https://www.crummy.com/software/BeautifulSoup/bs4/doc/), Real Python: "Web Scraping with BeautifulSoup" (https://realpython.com).
- **Outcome:** Your website passes its own SEO audit with no critical issues.

6.11 Week 10 – Keyword Tracking and Uptime Monitoring

- **Study:** pandas DataFrames. Reading/writing Excel files. The `schedule` library. Windows Task Scheduler.
- **Task:** Run Script 1 (Keyword Tracker) and record your initial rankings. Set up Script 4 (Uptime Monitor) to run continuously. Schedule the keyword tracker to run weekly using Task Scheduler.
- **Resources:** Pandas getting started (https://pandas.pydata.org/docs/getting_started/), schedule library docs (https://schedule.readthedocs.io).
- **Outcome:** First keyword ranking data recorded. Uptime monitor running.

6.12 Week 11 – LinkedIn Content and Community

- **Study:** LinkedIn content best practices. Writing for professional audiences. Hashtag strategy. Using LinkedIn analytics.
- **Task:** Publish the first month of content using your calendar from Chapter 4. Post 3 times this week. Join 2 LinkedIn groups. Leave 5 thoughtful comments on others' posts.
- **Resources:** LinkedIn's own Creator resources (https://www.linkedin.com/creators/), Search YouTube: "LinkedIn content strategy 2025".
- **Outcome:** First 3 posts published. First engagement (likes/comments) received.

6.13 Week 12 – Review, Improve, and Plan Ahead

- **Study:** Google Search Console data review. Understanding impressions, clicks, CTR, and average position. Identifying improvement opportunities.
- **Task:** Check Google Search Console for your first performance data. Identify your top 3 ranking keywords. Compare keyword tracker data from Weeks 10 and 12. Write a short plan for the next 4 weeks based on what you observe.
- **Resources:** Google Search Console tutorial on YouTube, Ahrefs YouTube channel (free SEO education).
- **Outcome:** First real Google ranking data. A written plan for the next phase of growth.

6.14 After Week 12: Continuing the Journey

- **Add blog posts:** A blog is the best long-term SEO strategy. Write one 800-word article per month on tax topics relevant to your city. E.g., "GST Registration for Small Businesses in [City]: A Complete Guide".
- **Build backlinks:** Ask local news websites, business associations, and chambers of commerce to link to your father's website.
- **Learn Django:** Once you are comfortable with Flask, Django will let you add more complex features (user accounts, blog CMS, etc.).
- **Learn Google Analytics 4:** Install it on your website for detailed visitor behaviour tracking.
- **Explore the Google Search Console API:** Automate ranking data extraction directly from Google (more reliable than the googlesearch script).

What You Learned

- A structured 12-week plan prevents overwhelm and ensures steady progress.
- The first 4 weeks focus on building and deploying the website.
- Weeks 5–8 focus on SEO setup and LinkedIn.
- Weeks 9–10 introduce Python automation.
- Weeks 11–12 focus on content creation and reviewing results.
- SEO results take 3–6 months to show — start early, be consistent.

Practice Task

1. Print this roadmap (or save the PDF from Overleaf) and pin it where you study.

2. Mark today's date on Week 1. Calculate your target date for Week 12.
3. Set a recurring 1-hour daily reminder on your phone: "Work on Dad's website".
4. Share your weekly progress with a friend or mentor to stay accountable.

Conclusion: You Have Everything You Need

You now have a complete blueprint. Let us be clear about what you are actually capable of:

- You can build a professional, multi-page website from scratch using Flask.
- You understand how Google ranks websites and what to do about it.
- You know how to set up your father as a local authority on LinkedIn.
- You can write Python scripts that automate real SEO tasks.
- You have a 12-week plan that tells you exactly what to do each week.

This is not a small thing. Most adults — let alone Class 12 students — cannot do any of this. You will be able to do all of it.

The only thing between you and results is **consistent action**. Open your Anaconda Prompt today. Activate your `webdev` environment. Start with Week 1. The rest will follow.

Final Tip: When You Get Stuck

Everyone gets stuck. That is not failure — it is part of learning. When you are stuck:

1. Re-read the relevant section of this book.
2. Copy your exact error message into Google and search for it.
3. Ask on Stack Overflow (https://stackoverflow.com).
4. Use the official documentation for whatever tool you are working with.
5. Ask ChatGPT or Claude to explain the specific error you are seeing.

You will solve it. Every professional developer does exactly this.

Good luck, Minza. Make your father proud.